

GUÍA DE ESTUDIO

Sistemas de Numeración

Basada en: Diseño Digital — M. Morris Mano, 3.^a Edición

Capítulo 1: Sistemas Binarios

Autor: Ing. Jorge N. Mercado

Contenidos

1. Representación posicional y forma polinomial
2. Sistema Binario (base 2)
3. Sistema Octal (base 8)
4. Sistema Hexadecimal (base 16)
5. Conversiones entre bases con parte fraccionaria
6. Complementos a la base y a la base disminuida
7. Números binarios con signo
8. Códigos binarios: BCD, Gray, ASCII
9. Ejercicios resueltos y propuestos

Ingeniería Electrónica — 3.^{er} Año

UTN-FRC

Índice

1. Introducción a los Sistemas de Numeración	4
2. Forma Polinomial de los Números	4
2.1. Definición General	4
2.2. Forma Polinomial Decimal (base 10)	4
2.3. Forma Polinomial Binaria (base 2)	5
2.4. Tabla de Potencias de 2	5
3. Conversiones Entre Sistemas de Numeración	6
3.1. Cualquier Base $\rightarrow$ Decimal (Expansión Polinomial)	6
3.2. Decimal $\rightarrow$ Base r — Parte Entera	6
3.3. Decimal $\rightarrow$ Base r — Parte Fraccionaria	7
4. Conversiones Rápidas: Binario $\leftrightarrow$ Octal $\leftrightarrow$ Hexadecimal	8
4.1. Tabla de Equivalencias (0–15)	8
4.2. Binario $\leftrightarrow$ Octal (grupos de 3 bits)	8
4.3. Binario $\leftrightarrow$ Hexadecimal (grupos de 4 bits)	9
5. Complementos	9
5.1. Complemento a la Base Disminuida — C_{r-1}	9
5.2. Complemento a la Base — C_r	10
6. Resta con Complementos	11
7. Números Binarios con Signo	12
7.1. Magnitud con Signo	12
7.2. Complemento a Uno con Signo	12
7.3. Complemento a Dos con Signo (estándar)	12
7.4. Tabla Comparativa (4 bits con signo)	13
7.5. Suma y Resta en Complemento a Dos	13
8. Códigos Binarios	13
8.1. Código BCD (Binary Coded Decimal)	14
8.2. Código Gray	14
8.3. Código ASCII	15
9. Resumen de Métodos de Conversión	16
10. Ejercicios Resueltos	16
10.1. Ejercicio 10.1 — Conversión completa con fracción	16
10.2. Ejercicio 10.2 — Complemento a Dos con fracción	17
10.3. Ejercicio 10.3 — Resta con Complemento a Dos y fracción	17

10.4. Ejercicio 10.4 — Suma BCD	17
11.Ejercicios Propuestos	18
12.Glosario de Términos Clave	20
13.Formulario de Referencia Rápida	20

1. Introducción a los Sistemas de Numeración

En los sistemas digitales, toda la información se representa internamente mediante señales eléctricas que sólo admiten dos valores: presencia o ausencia de tensión. Esta dualidad es la razón por la que el **sistema binario (base 2)** es fundamental en la electrónica digital. Sin embargo, dado que los seres humanos utilizamos el sistema decimal (base 10), es imprescindible comprender las reglas de representación y conversión entre distintas bases numéricas.

Un **sistema de numeración posicional de base r** está caracterizado por:

- Un conjunto de r símbolos (dígitos) con valores de 0 a $r - 1$.
- Un *valor posicional*: cada posición representa una potencia de r .
- La posición a la izquierda del separador decimal corresponde a potencias positivas; la derecha, a potencias negativas.

Bases más utilizadas en Ingeniería Digital

Base 2	(Binario)	dígitos $\{0, 1\}$	representación en hardware
Base 8	(Octal)	dígitos $\{0, \dots, 7\}$	3 bits por dígito
Base 10	(Decimal)	dígitos $\{0, \dots, 9\}$	interfaz humana habitual
Base 16	(Hexadecimal)	dígitos $\{0, \dots, 9, A, \dots, F\}$	4 bits por dígito

2. Forma Polinomial de los Números

2.1 Definición General

Cualquier número N expresado en base r se puede escribir como la suma de sus coeficientes multiplicados por la potencia correspondiente de r :

$$N_{(r)} = a_n r^n + a_{n-1} r^{n-1} + \dots + a_1 r + a_0 + a_{-1} r^{-1} + a_{-2} r^{-2} + \dots + a_{-m} r^{-m} \quad (1)$$

donde a_j son los coeficientes (dígitos) con $0 \leq a_j \leq r - 1$, y j es el índice de posición (positivo para la parte entera, negativo para la parte fraccionaria).

2.2 Forma Polinomial Decimal (base 10)

El sistema decimal tiene base $r = 10$ y coeficientes $\{0, 1, 2, \dots, 9\}$. La expansión polinomial del número 4835,27 es:

$$\begin{aligned}
 (4835,27)_{10} &= 4 \cdot 10^3 + 8 \cdot 10^2 + 3 \cdot 10^1 + 5 \cdot 10^0 + 2 \cdot 10^{-1} + 7 \cdot 10^{-2} \\
 &= 4000 + 800 + 30 + 5 + 0,2 + 0,07 \\
 &= 4835,27
 \end{aligned}$$

Nota: a la izquierda del separador decimal están las potencias positivas de 10; a la derecha, las negativas.

2.3 Forma Polinomial Binaria (base 2)

El sistema binario usa sólo los coeficientes $\{0, 1\}$. Cada posición vale una potencia de 2. La conversión a decimal se realiza directamente por expansión polinomial.

Ejemplo 2.1 — Binario a Decimal: $(11010,11)_2$

$$\begin{aligned}
 (11010,11)_2 &= 1 \cdot 2^4 + 1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0 + 1 \cdot 2^{-1} + 1 \cdot 2^{-2} \\
 &= 16 + 8 + 0 + 2 + 0 + 0,5 + 0,25 \\
 &= (26,75)_{10} \quad \checkmark
 \end{aligned}$$

Regla práctica: para la parte entera, sume las potencias de 2 correspondientes a los bits 1. Para la parte fraccionaria: $2^{-1} = 0,5$; $2^{-2} = 0,25$; $2^{-3} = 0,125$; $2^{-4} = 0,0625$; etc.

2.4 Tabla de Potencias de 2

n	2^n	n	2^n	n	2^n
0	1	8	256	16	65 536
1	2	9	512	17	131 072
2	4	10	1 024	18	262 144
3	8	11	2 048	19	524 288
4	16	12	4 096	20	1 048 576 (1 M)
5	32	13	8 192	21	2 097 152
6	64	14	16 384	22	4 194 304
7	128	15	32 768	23	8 388 608

Prefijos en informática: $2^{10} = 1 \text{ K}$; $2^{20} = 1 \text{ M}$; $2^{30} = 1 \text{ G}$; $2^{40} = 1 \text{ T}$.

3. Conversiones Entre Sistemas de Numeración

3.1 Cualquier Base $\rightarrow$ Decimal (Expansión Polinomial)

Método universal: expandir en potencias de la base y sumar todos los términos (ecuación 1).

Ejemplo 3.1-A — Octal a Decimal: $(127.4)_8$

$$(127.4)_8 = 1 \cdot 8^2 + 2 \cdot 8^1 + 7 \cdot 8^0 + 4 \cdot 8^{-1} = 64 + 16 + 7 + 0.5 = (87.5)_{10}$$

Ejemplo 3.1-B — Hexadecimal a Decimal: $(B65F)_{16}$

$$(B65F)_{16} = 11 \cdot 16^3 + 6 \cdot 16^2 + 5 \cdot 16^1 + 15 \cdot 16^0 = 45056 + 1536 + 80 + 15 = (46687)_{10}$$

(recordar: B= 11, F= 15)

Ejemplo 3.1-C — Base 5 con fracción: $(4021.2)_5$

$$(4021.2)_5 = 4 \cdot 5^3 + 0 \cdot 5^2 + 2 \cdot 5^1 + 1 \cdot 5^0 + 2 \cdot 5^{-1} = 500 + 0 + 10 + 1 + 0.4 = (511.4)_{10}$$

3.2 Decimal $\rightarrow$ Base r — Parte Entera

Algoritmo para la Parte Entera

1. Dividir el número entero entre r .
2. Registrar el residuo: es el coeficiente a_0 (posición menos significativa, LSB).
3. Repetir con el cociente hasta que el cociente sea 0.
4. Leer los residuos **de abajo hacia arriba** (del último obtenido al primero).

Ejemplo 3.2-A — Decimal a Binario: $(41)_{10}$

Número	$\div 2$	Residuo
41	20	1 $\leftarrow a_0$ (LSB)
20	10	0
10	5	0
5	2	1
2	1	0
1	0	1 $\leftarrow a_5$ (MSB)

Leer residuos de abajo a arriba: $(101001)_2$

Verificación: $32 + 8 + 1 = 41$ ✓

Ejemplo 3.2-B — Decimal a Octal: $(153)_{10}$

$$\begin{array}{r|l|l} 153 & 19 & 1 \\ 19 & 2 & 3 \\ 2 & 0 & 2 \end{array} \Rightarrow (231)_8$$

Verificación: $2 \cdot 64 + 3 \cdot 8 + 1 = 128 + 24 + 1 = 153 \quad \checkmark$

3.3 Decimal $\rightarrow$ Base r — Parte Fraccionaria

Algoritmo para la Parte Fraccionaria

1. Multiplicar la fracción decimal por r .
2. Registrar la parte entera del resultado: es el coeficiente a_{-1} (posición más significativa de la fracción).
3. Repetir con la parte fraccionaria restante hasta obtener fracción 0 o la precisión deseada.
4. Leer los enteros **de arriba hacia abajo**.

Ejemplo 3.2-C — Fracción Decimal a Binario: $(0.6875)_{10}$

Fracción	$\times 2$	Entero
0,6875	1,3750	1 $\leftarrow a_{-1}$
0,3750	0,7500	0
0,7500	1,5000	1
0,5000	1,0000	1 $\leftarrow a_{-4}$ (fracción = 0, termina)

Leer enteros de arriba a abajo: $(0,1011)_2$

Verificación: $0,5 + 0 + 0,125 + 0,0625 = 0,6875 \quad \checkmark$

Ejemplo 3.2-D — Número completo: $(41.6875)_{10}$ a Binario

- Parte entera: $(41)_{10} \rightarrow (101001)_2$ (Ejemplo 3.2-A)
- Parte fraccionaria: $(0,6875)_{10} \rightarrow (0,1011)_2$ (Ejemplo 3.2-C)

$$\therefore (41,6875)_{10} = (101001,1011)_2$$

Ejemplo 3.2-E — Fracción Decimal a Octal: $(0.513)_{10}$

$$\begin{array}{r|l|l} 0,513 & \times 8 = 4,104 & 4 \\ 0,104 & \times 8 = 0,832 & 0 \\ 0,832 & \times 8 = 6,656 & 6 \\ 0,656 & \times 8 = 5,248 & 5 \\ 0,248 & \times 8 = 1,984 & 1 \\ 0,984 & \times 8 = 7,872 & 7 \end{array} \Rightarrow (0,406517\dots)_8$$

Número completo: $(153,513)_{10} = (231,406517\dots)_8$

Advertencia: Fracciones Periódicas

No toda fracción decimal tiene representación *finita* en otra base.

Por ejemplo, $0,1_{(10)}$ es periódica en binario: $(0,0\overline{0011})_2$

Esto es análogo a $1/3 = 0,\overline{3}$ en decimal.

En la práctica se trabaja con la precisión necesaria (n bits o dígitos).

4. Conversiones Rápidas: Binario $\leftrightarrow$ Octal $\leftrightarrow$ Hexadecimal

Dado que $2^3 = 8$ y $2^4 = 16$, existe una correspondencia directa entre grupos de bits y dígitos octales/hexadecimales.

4.1 Tabla de Equivalencias (0–15)

Decimal	Binario (4 bits)	Octal	Hexadecimal
0	0000	0	0
1	0001	1	1
2	0010	2	2
3	0011	3	3
4	0100	4	4
5	0101	5	5
6	0110	6	6
7	0111	7	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F

4.2 Binario $\leftrightarrow$ Octal (grupos de 3 bits)

Agrupar los bits en grupos de 3 partiendo del separador decimal hacia ambos lados. Añadir ceros de relleno si es necesario.

Ejemplo 4.2-A — Binario a Octal

$$\underbrace{010}_2 \underbrace{110}_6 \underbrace{001}_1 \underbrace{101}_5 \underbrace{011}_3 . \underbrace{111}_7 \underbrace{100}_4 \underbrace{000}_0 \underbrace{110}_6 \Rightarrow (26153,7406)_8$$

Ejemplo 4.2-B — Octal a Binario: $(673.124)_8$

$$6 \rightarrow 110, \quad 7 \rightarrow 111, \quad 3 \rightarrow 011 \quad . \quad 1 \rightarrow 001, \quad 2 \rightarrow 010, \quad 4 \rightarrow 100$$

$$\Rightarrow (110\ 111\ 011,001\ 010\ 100)_2$$

4.3 Binario $\leftrightarrow$ Hexadecimal (grupos de 4 bits)**Ejemplo 4.3-A — Binario a Hexadecimal**

$$\underbrace{0010}_2 \underbrace{1100}_C \underbrace{0110}_6 \underbrace{1011}_B . \underbrace{1111}_F \underbrace{0010}_2 \Rightarrow (2C6B,F2)_{16}$$

Ejemplo 4.3-B — Hexadecimal a Binario: $(306.D)_{16}$

$$3 \rightarrow 0011, \quad 0 \rightarrow 0000, \quad 6 \rightarrow 0110 \quad . \quad D \rightarrow 1101 \Rightarrow (0011\ 0000\ 0110,1101)_2$$

Ventaja del Hexadecimal

Un byte (8 bits) se representa exactamente con **2 dígitos hexadecimales**.

Ejemplo: $11001010_2 = CA_{16}$ (sólo 2 caracteres vs. 8 dígitos binarios).

Por eso los manuales de computadoras prefieren la notación hexadecimal.

5. Complementos

Los complementos se utilizan en sistemas digitales para simplificar la operación de resta y para representar números negativos. Para una base r se definen dos tipos.

5.1 Complemento a la Base Disminuida — C_{r-1} **Definición**

Dado un número N de n dígitos en base r :

$$C_{r-1}(N) = (r^n - 1) - N$$

Base 10 $\rightarrow$ complemento a **nueve**: restar cada dígito de 9

Base 2 $\rightarrow$ complemento a **uno**: invertir cada bit ($0 \leftrightarrow 1$)

Base 16 $\rightarrow$ complemento a **15 (F)**: restar cada dígito de F

Ejemplos de Complemento a Nueve (decimal)

$$N = 546700 \ (n=6) : \quad C_9(N) = 999999 - 546700 = 453299$$

$$N = 012398 \ (n=6) : \quad C_9(N) = 999999 - 012398 = 987601$$

Ejemplos de Complemento a Uno (binario)

$$N = 1011000 : \quad C_1(N) = 0100111 \quad (\text{invertir cada bit})$$

$$N = 0101101 : \quad C_1(N) = 1010010$$

5.2 Complemento a la Base — C_r **Definición**

$$C_r(N) = r^n - N \quad (N \neq 0), \quad C_r(0) = 0$$

Relación práctica: $C_r(N) = C_{r-1}(N) + 1$

Complemento a DOS Método 1: $C_1(N) + 1$

Método 2 (directo): copiar bits de derecha a izquierda hasta el primer 1 incluido; luego invertir el resto.

Ejemplos de Complemento a Dos (binario)

$$N = 1101100 : \quad C_1 = 0010011 \rightarrow C_2 = 0010100$$

Método directo: copiar 00, copiar 1er 1 $\rightarrow$ 100,
invertir 1101 $\rightarrow$ 0010 $\Rightarrow$ 0010100

$$N = 0110111 : \quad C_1 = 1001000 \rightarrow C_2 = 1001001$$

Complemento a Diez (decimal)

$$N = 012398 : \quad C_9 = 987601 \rightarrow C_{10} = 987602$$

$$N = 246700 :$$

Copiar 00, $10 - 7 = 3 \rightarrow 300$, $9 - 6 = 3$, $9 - 4 = 5$, $9 - 2 = 7$
 $\Rightarrow C_{10} = 753300$

Propiedad: el complemento del complemento restaura el original

$$C_r(C_r(N)) = r^n - (r^n - N) = N$$

6. Resta con Complementos

Los complementos permiten convertir la resta en una suma, simplificando la circuitería del hardware digital.

Algoritmo de Resta $M - N$ con Complemento a r

1. Calcular $C_r(N)$.
2. Calcular $S = M + C_r(N)$.
3. **Hay acarreo final:** $M \geq N$, resultado positivo. Descartar el acarreo; la respuesta es $S - r^n$.
4. **No hay acarreo final:** $M < N$, resultado negativo. La respuesta es $-C_r(S)$.

Ejemplo 6-A — Con C_{10} : $(72532 - 3250)$

$$\begin{array}{r} 72532 \\ [C_{10}(03250)] + 96750 \\ \hline 169282 \end{array} \rightarrow \text{Descartar } 10^5 : \boxed{69282} \quad \checkmark$$

Ejemplo 6-B — Con C_{10} : $(3250 - 72532)$

$$\begin{array}{r} 03250 \\ [C_{10}(72532)] + 27468 \\ \hline 30718 \end{array} \rightarrow \text{Sin acarreo} \Rightarrow -C_{10}(30718) = \boxed{-69282} \quad \checkmark$$

Ejemplo 6-C — Con C_2 : $X=1010100, Y=1000011$

a) $X - Y$:

$$\begin{array}{r} 1010100 \\ [C_2(1000011)] + 0111101 \\ \hline 10010001 \end{array} \rightarrow \text{Descartar } 2^7 : \boxed{0010001 = 17_{(10)}} \quad \checkmark$$

b) $Y - X$:

$$\begin{array}{r} 1000011 \\ [C_2(1010100)] + 0101100 \\ \hline 1101111 \end{array} \rightarrow \text{Sin acarreo} \Rightarrow -C_2(1101111) = \boxed{-0010001} \quad \checkmark$$

7. Números Binarios con Signo

Los números binarios con signo utilizan el bit de posición más significativa (MSB) para indicar el signo: **0** = **positivo**, **1** = **negativo**. Existen tres representaciones principales.

7.1 Magnitud con Signo

El MSB es el bit de signo y los $n - 1$ bits restantes representan la magnitud en binario puro.

```
+9 --> 0 0001001 (8 bits: bit de signo 0, magnitud 9)
-9 --> 1 0001001 (bit de signo 1, magnitud 9)
```

7.2 Complemento a Uno con Signo

Los números negativos se obtienen complementando a uno *todos* los bits (incluido el bit de signo) del número positivo equivalente.

```
+9 --> 00001001
-9 --> 11110110 (complemento a uno de 00001001)
```

7.3 Complemento a Dos con Signo (estándar)

Los números negativos se obtienen calculando el C_2 del número positivo equivalente. Este es el estándar en todos los procesadores modernos: única representación del cero y circuitería aritmética más simple.

```
+9 --> 00001001
-9 --> 11110111 (complemento a dos de 00001001)
```

7.4 Tabla Comparativa (4 bits con signo)

Decimal	C_2 con signo	C_1 con signo	Magnitud con signo
+7	0111	0111	0111
+6	0110	0110	0110
+5	0101	0101	0101
+4	0100	0100	0100
+3	0011	0011	0011
+2	0010	0010	0010
+1	0001	0001	0001
0	0000	0000	0000
-0	— (no existe)	1111	1000
-1	1111	1110	1001
-2	1110	1101	1010
-3	1101	1100	1011
-4	1100	1011	1100
-5	1011	1010	1101
-6	1010	1001	1110
-7	1001	1000	1111
-8	1000	—	—

7.5 Suma y Resta en Complemento a Dos

Regla de Suma en C_2 con Signo

Sumar los n bits incluyendo el bit de signo. Descartar cualquier acarreo que salga del bit de signo. El resultado negativo ya está automáticamente en forma de C_2 .

Ejemplos de Suma (8 bits)

$$(+6) + (+13): \quad 00000110 + 00001101 = 00010011 = +19 \quad \checkmark$$

$$(-6) + (+13): \quad 11111010 + 00001101 = 00000111 = +7 \quad \checkmark$$

$$(+6) + (-13): \quad 00000110 + 11110011 = 11111001 = -7 \quad \checkmark$$

$$(-6) + (-13): \quad 11111010 + 11110011 = 11101101 = -19 \quad \checkmark$$

8. Códigos Binarios

Los sistemas digitales no sólo manipulan números sino también otros elementos discretos. Los **códigos binarios** asignan combinaciones de bits a cada símbolo del conjunto a representar.

8.1 Código BCD (Binary Coded Decimal)

BCD utiliza **4 bits** para representar cada dígito decimal del 0 al 9. Las 6 combinaciones 1010–1111 son inválidas en BCD.

Decimal	BCD (8421)	Decimal	BCD (8421)
0	0000	5	0101
1	0001	6	0110
2	0010	7	0111
3	0011	8	1000
4	0100	9	1001

Ejemplo BCD — Número decimal 396

$$3 \rightarrow 0011, \quad 9 \rightarrow 1001, \quad 6 \rightarrow 0110$$

$$(396)_{10} = \underbrace{0011}_3 \underbrace{1001}_9 \underbrace{0110}_6 \text{ (BCD, 12 bits)} \text{ vs. } (396)_{10} = 110001100_2 \text{ (bin., 9 bits)}$$

Suma BCD: si la suma de dos dígitos BCD supera 9 (o produce acarreo), se suma $0110_2 = 6_{10}$ para obtener el dígito BCD correcto.

Suma BCD: $184 + 576 = 760$

$$\begin{array}{r}
 \text{Acarreo BCD:} \quad \quad 1 \quad \quad 1 \\
 184 = \quad 0001 \quad 1000 \quad 0100 \\
 576 = \quad 0101 \quad 0111 \quad 0110 \\
 \hline
 \text{Suma binaria:} \quad 0111 \quad 1111 \quad 1010 \\
 \text{Corrección:} \quad \quad \quad +0110 \quad +0110 \\
 \hline
 \text{BCD:} \quad 0111 \quad 0110 \quad 0000 \\
 \quad \quad \quad 7 \quad \quad 6 \quad 0 \\
 \Rightarrow \quad (760)_{BCD} = 0111 \ 0110 \ 0000 \quad \checkmark
 \end{array}$$

8.2 Código Gray

El código Gray tiene la propiedad de que entre dos números *consecutivos* cualesquiera sólo cambia **un bit**. Es fundamental en convertidores analógico-digital (ADC) y encoders rotativos.

Dec	Binario	Gray	Dec	Binario	Gray
0	0000	0000	8	1000	1100
1	0001	0001	9	1001	1101
2	0010	0011	10	1010	1111
3	0011	0010	11	1011	1110
4	0100	0110	12	1100	1010
5	0101	0111	13	1101	1011
6	0110	0101	14	1110	1001
7	0111	0100	15	1111	1000

Al pasar de 7 a 8 en Gray sólo cambia 1 bit (0100→1100), mientras que en binario puro cambian los 4 bits (0111→1000).

8.3 Código ASCII

ASCII (*American Standard Code for Information Interchange*) utiliza **7 bits** para codificar 128 caracteres.

Carácter	ASCII (7 bits)	Hex	Decimal
'A'	1000001	41	65
'a'	1100001	61	97
'0'	0110000	30	48
'9'	0111001	39	57
SPACE	0100000	20	32

Las computadoras modernas almacenan un carácter ASCII en 1 byte (8 bits). El octavo bit puede usarse como **bit de paridad** para detección de errores.

9. Resumen de Métodos de Conversión

Tabla Resumen

De cualquier base a DECIMAL Expansión polinomial: sumar $a_j \cdot r^j$

De DECIMAL a base r :

Parte entera

Divisiones sucesivas entre r (leer residuos de abajo a arriba)

Parte fraccionaria

Multiplicaciones sucesivas por r (leer enteros de arriba a abajo)

Binario $\rightarrow$ Octal

Agrupar bits de **3 en 3** desde el punto decimal

Octal $\rightarrow$ Binario

Expandir cada dígito octal en 3 bits

Binario $\rightarrow$ Hex

Agrupar bits de **4 en 4** desde el punto decimal

Hex $\rightarrow$ Binario

Expandir cada dígito hex en 4 bits

Octal $\leftrightarrow$ Hex

Pasar por binario como intermediario

10. Ejercicios Resueltos

10.1 Ejercicio 10.1 — Conversión completa con fracción

Enunciado: Convertir $(34,4375)_{10}$ a binario, octal y hexadecimal.

Paso 1: Decimal $\rightarrow$ Binario

Parte entera (34):

$$\begin{array}{r|l|l}
 34 & 17 & 0 \\
 17 & 8 & 1 \\
 8 & 4 & 0 \\
 4 & 2 & 0 \\
 2 & 1 & 0 \\
 1 & 0 & 1
 \end{array} \Rightarrow (100010)_2$$

Parte fraccionaria (0,4375):

$$\begin{array}{r|l|l}
 0,4375 & 0,875 & 0 \\
 0,8750 & 1,750 & 1 \\
 0,7500 & 1,500 & 1 \\
 0,5000 & 1,000 & 1
 \end{array} \Rightarrow (0,0111)_2$$

$$\therefore (34,4375)_{10} = (100010,0111)_2$$

Paso 2: Binario → Octal (grupos de 3)

$$\underbrace{100}_4 \underbrace{010}_2 \cdot \underbrace{011}_3 \underbrace{100}_4 \Rightarrow (34,4375)_{10} = (42,34)_8$$

Paso 3: Binario → Hexadecimal (grupos de 4)

$$\underbrace{0010}_2 \underbrace{0010}_2 \cdot \underbrace{0111}_7 \underbrace{0000}_0 \Rightarrow (34,4375)_{10} = (22,7)_{16}$$

10.2 Ejercicio 10.2 — Complemento a Dos con fracción

Enunciado: Obtener el C_2 de $(101100,110)_2$.

Solución

1. Quitar el punto: 101100110 ($n = 9$ bits).
2. C_1 : 010011001
3. C_2 : $010011001 + 1 = 010011010$
4. Recolocar el punto: 010011.010
5. **Verificación:** $101100.110 + 010011.010 = 1\ 000000.000$ (acarreo en bit $n + 1$)
✓

10.3 Ejercicio 10.3 — Resta con Complemento a Dos y fracción

Enunciado: Calcular $(1011,101)_2 - (0110,110)_2$.

Solución

$$C_2(0110.110) = C_1(0110.110) + 1 = 1001.001 + 1 = 1001.010$$

$$\begin{array}{r} 1011.101 \\ +1001.010 \\ \hline 10100.111 \end{array} \rightarrow \text{Descartar acarreo } 2^4 : \boxed{0100.111}$$

Verificación: $11,625_{(10)} - 6,75_{(10)} = 4,875_{(10)}$; $(0100,111)_2 = 4 + 0,5 + 0,25 + 0,125 = 4,875$ ✓

10.4 Ejercicio 10.4 — Suma BCD

Enunciado: Calcular $184 + 576$ en BCD.

Solución columna por columna

Unidades ($4+6 = 10$): $0100+0110 = 1010 > 9 \rightarrow$ sumar 0110: $1010+0110 = 10000$
 $\Rightarrow$ suma= 0000, acarreo= 1

Decenas ($8 + 7 + 1 = 16$): $1000 + 0111 + 0001 = 10000 \Rightarrow \text{suma} = 0000$, sumar $0110 = 0110$, acarreo = 1

Centenas ($1 + 5 + 1 = 7$): $0001 + 0101 + 0001 = 0111 \leq 9 \Rightarrow \text{sin corrección}$

$$\therefore (760)_{BCD} = 0111\ 0110\ 0000 \quad \checkmark$$

11. Ejercicios Propuestos

Los ejercicios están organizados por nivel de dificultad. Se recomienda resolver cada nivel antes de avanzar al siguiente.

Nivel 1 — Conversiones básicas

1. Convertir a decimal usando forma polinomial: $(10110,0101)_2$, $(16,5)_{16}$, $(26,24)_8$.
2. Convertir $(345)_{10}$ a binario por divisiones sucesivas.
3. Convertir $(345)_{10}$ a hexadecimal y luego de hex a binario. ¿Qué método es más rápido?
4. Convertir $68BE_{16}$ a binario y de binario a octal.
5. Listar los números octales y hexadecimales del 16 al 32.

Nivel 2 — Fracciones y conversiones combinadas

6. Convertir $(0,3125)_{10}$ a binario (¿termina?) y luego a hexadecimal.
7. Convertir $(34,4375)_{10}$ a: a) binario, b) octal, c) hexadecimal.
8. Calcular el equivalente binario de $1/3$ hasta 8 posiciones. Convertir el resultado a decimal. ¿Cuánto difiere de $1/3$?
9. Convertir $(1,11010)_2$ y $(1110,10)_2$ a decimal y hexadecimal. Explicar por qué el decimal de (b) es 8 veces el de (a).
10. Determinar la base si: a) $14 \div 2 = 5$; b) $54 \div 4 = 13$; c) $24 + 17 = 40$.

Nivel 3 — Complementos

11. Calcular C_9 y C_{10} de: 98127634, 72049900, 10000000.
12. Calcular C_1 y C_2 de: 11101010, 01111110, 10000000.
13. Obtener el C_{16} de AF3B; luego convertir AF3B a binario y comparar el C_2 con el C_{16} .
14. Efectuar con C_{10} : a) $7188 - 3049$; b) $150 - 2100$; c) $2997 - 7992$; d) $1321 - 375$.
15. Efectuar con C_2 : a) $11011 - 11001$; b) $110100 - 10101$; c) $1011 - 110000$.

Nivel 4 — Aritmética con signo

16. Representar $+61$ y -27 en C_2 con 8 bits. Realizar: $(+27) + (-61)$, $(-27) + (+61)$, $(-27) + (-61)$.
17. Los números $+9826$ y -801 están en C_{10} con signo. Efectuar las cuatro combinaciones de suma. (La suma $+9826 + 801 = +10627$ requiere 6 dígitos).
18. ¿Cuántos bytes hay en un sistema de 32 K, 64 M y 6,4 G bytes? Expresar en potencias de 2.

Respuestas parciales para autoevaluación

1. $22,3125_{(10)}$; $22,5_{(10)}$; $22,3125_{(10)}$
2. $(101011001)_2$
4. $68BE_{16} = 0110\ 1000\ 1011\ 1110_2 = 064276_8$
6. $0,3125 = (0,0101)_2 = (0,5)_{16}$
7. $34,4375_{(10)} = 100010,0111_{(2)} = 42,34_{(8)} = 22,7_{(16)}$
10. a) base 3; b) base 7; c) base 5
11. $C_9(98127634) = 01872365$; $C_{10} = 01872366$
- 14a. $7188 - 3049 = 4139$; [14b.] $150 - 2100 = -1950$

12. Glosario de Términos Clave

Término	Definición
Base (Radix)	Número de símbolos distintos usados en un sistema posicional. Determina el valor de cada posición.
Bit	Dígito binario (<i>binary digit</i>). Puede valer 0 o 1. Unidad mínima de información digital.
Byte	Grupo de 8 bits. Puede representar un carácter ASCII o un número de 0 a 255 (sin signo).
MSB	Bit de mayor peso (<i>Most Significant Bit</i>). En números con signo, indica el signo.
LSB	Bit de menor peso (<i>Least Significant Bit</i>), posición de las unidades.
Complemento a uno	Se obtiene invirtiendo todos los bits. $C_1(N) = 2^n - 1 - N$.
Complemento a dos	Se obtiene sumando 1 al complemento a uno. $C_2(N) = 2^n - N$. Estándar para negativos.
BCD	<i>Binary Coded Decimal</i> . Codifica cada dígito decimal con 4 bits (0000–1001).
Código Gray	Código donde dos valores consecutivos difieren en sólo 1 bit. Reduce errores en transiciones.
ASCII	<i>American Standard Code for Information Interchange</i> . 7 bits, 128 caracteres.
Acarreo (Carry)	Transporte a la posición de mayor peso cuando la suma supera el valor máximo del dígito.
Desbordamiento	Ocurre cuando el resultado supera la capacidad de representación (n bits).
Expansión polinomial	Suma de los coeficientes multiplicados por las potencias de la base. Método universal de conversión a decimal.

13. Formulario de Referencia Rápida

Forma Polinomial General

$$N_{(r)} = \sum_{j=-m}^n a_j \cdot r^j, \quad 0 \leq a_j \leq r - 1$$

Potencias Fraccionarias de 2

$$2^{-1} = 0,5 \quad 2^{-2} = 0,25 \quad 2^{-3} = 0,125 \quad 2^{-4} = 0,0625 \quad 2^{-5} = 0,03125 \quad 2^{-6} = 0,015625$$

Complementos

$$C_{r-1}(N) = (r^n - 1) - N$$

Regla: restar cada dígito de $(r - 1)$

$$C_r(N) = r^n - N$$

Regla: $C_{r-1}(N) + 1$

Complemento a UNO binario: invertir cada bit ($0 \leftrightarrow 1$)

Complemento a DOS binario: copiar desde LSB hasta el primer 1 incluido; luego invertir.

Resta con Complementos

$$M - N = M + C_r(N)$$

- **Hay acarreo final** $\Rightarrow$ resultado positivo, descartar el acarreo.
- **No hay acarreo** $\Rightarrow$ resultado negativo = $-C_r(\text{suma obtenida})$.

Suma en C_2 con Signo

Sumar los n bits (incluyendo el bit de signo).

Descartar cualquier acarreo que salga del bit de signo.

Desbordamiento: ambos operandos con igual signo producen resultado de signo contrario.

Correspondencia de Bits

$$3 \text{ bits} \longleftrightarrow 1 \text{ dígito octal} \quad (2^3 = 8)$$

$$4 \text{ bits} \longleftrightarrow 1 \text{ dígito hexadecimal} \quad (2^4 = 16)$$

$$8 \text{ bits (1 byte)} \longleftrightarrow 2 \text{ dígitos hexadecimales}$$